SUPPORT DE COURS · DÉVELOPPEMENT WEB

CSS

L'essentiel pour mettre en forme une page web

Par Harouna Kane



Le CSS, c'est quoi ?

Définition

CSS (Cascading Style Sheets, « feuilles de style en cascade ») est le langage qui **met en forme** une page HTML : couleurs, polices, tailles, marges, bordures, fonds et mise en page.

HTML	CSS
le contenu et la structure	**l'apparence**
titres, paragraphes, images, liens	couleurs, polices, tailles, placement

Où écrire le CSS ?

1 **Dans un fichier .css séparé** — Recommandé

```
<link rel="stylesheet" href="style.css">
```

2 **Dans le <head>, entre balises <style>** — À éviter

```
<style> p { color: red; } </style>
```

3 **Directement dans la balise** — À éviter

```
<p style="color: red;">
```

Bonne pratique : un seul fichier style.css, relié à toutes les pages du site. Une modification s'applique partout. Vérifier la compatibilité d'une propriété sur `caniuse.com`.

Relier le fichier et écrire une règle

Étape 1 : relier style.css à la page

```html
<!-- index.html -->
<head>
  <meta charset="UTF-8">
  <title>Mon site</title>
  <link rel="stylesheet" href="css/style.css">
</head>
```

Organisation des dossiers

```
mon-site/
├── index.html
├── css/
│   └── style.css
└── img/
```

Le chemin du href part du **fichier HTML**.

Étape 2 : écrire une règle




```css
p {
  color: #333333;        /* couleur du texte */
  font-size: 16px;       /* taille */
  margin-bottom: 12px;   /* espace dessous */
}
```

Cibler les éléments à styliser

Sélecteur	En CSS	Dans le HTML	Ce qui est ciblé
De balise	p { }	…	toutes les balises de la page
De classe	`.prix { }`	`<span class="prix">`	tous les éléments qui ont cette classe : réutilisable
D'identifiant	`#menu { }`	`<nav id="menu">`	un seul élément : un id est unique dans la page
Groupe	`h1, h2, h3 { }`	—	plusieurs sélecteurs, séparés par une virgule, même style
Universel	`* { }`	—	absolument tous les éléments

Styliser un seul mot avec

```
<p>Prix : <span class="prix">9 €</span></p>

.prix { color: green; font-weight: bold; }
```

Deux règles se contredisent : qui gagne ?

`style=""` **>** `#id` **>** `.classe` **>** `balise`

Le plus précis l'emporte. À précision égale, la dernière règle écrite gagne : c'est la « cascade ».

Combiner les sélecteurs

Syntaxe	Nom	Ce qui est ciblé
`div p`	Descendant (espace)	tous les <p> placés à l'intérieur d'une , à n'importe quelle profondeur
`div > p`	Enfant direct	les <p> qui sont enfants directs de la , pas les petits-enfants
`h2 + p`	Frère adjacent	le <p> qui suit immédiatement un <h2>
`h2 ~ p`	Frères suivants	tous les <p> placés après un <h2>, dans le même parent
`a[title]`	Attribut présent	les liens qui ont un attribut title
`input[type="email"]`	Attribut égal à	les champs dont l'attribut type vaut "email"

À retenir : ces syntaxes resserviront telles quelles en JavaScript, avec `document.querySelector("div > p")`. Pour s'entraîner : le jeu CSS Diner.

Pseudo-classes et pseudo-éléments

Pseudo-classes : un état

Un seul deux-points. L'élément est ciblé selon sa **situation** : survolé, visité, premier de la liste…

```
a:hover { }            /* survolé par la souris */
a:visited { }          /* lien déjà visité */
input:focus { }        /* champ sélectionné */
li:first-child { }     /* premier de la liste */
li:nth-child(2) { }    /* le 2e */
li:nth-child(odd) { }  /* 1er, 3e, 5e… */
p:not(.intro) { }      /* tous sauf .intro */
```

Pseudo-éléments :: une partie

Deux deux-points. On cible un **morceau** de l'élément, ou on en ajoute un.

```
p::first-line { }       /* 1re ligne */
p::first-letter { }     /* 1re lettre */
input::placeholder { } /* texte indicatif */
.titre::before {        /* ajoute avant */
  content: "→ ";
}
```

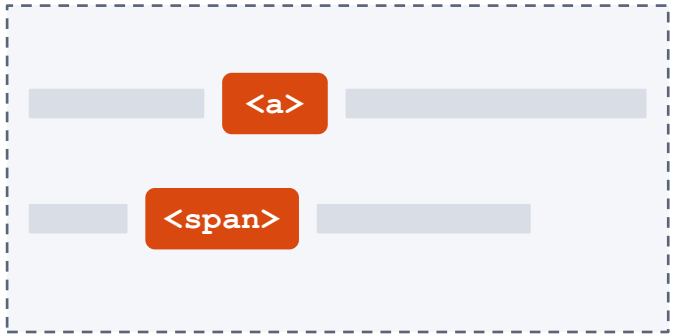
::before et **::after** ne s'affichent que si la propriété **content** est présente, même vide : content: "";

Chaque balise est une boîte



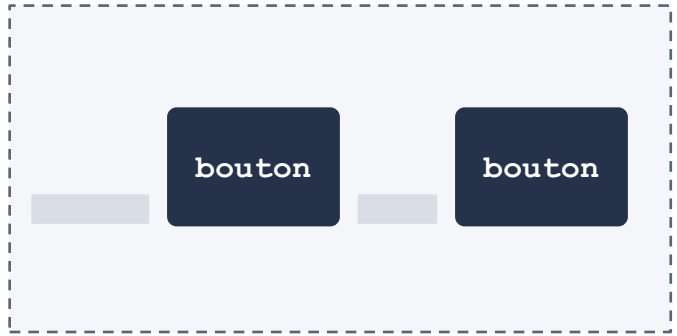
`display: block`

Prend toute la largeur. Retour à la ligne avant et après. Accepte width et height.
Ex. : div, p, h1, section, ul



`display: inline`

Prend la largeur de son contenu et reste dans la ligne. Ignore width et height.
Ex. : span, a, strong, em



`display: inline-block`

Reste dans la ligne comme un mot, mais accepte width, height et marges.
Ex. : boutons, liens d'un menu

On change le type d'une boîte avec **`display`**. Exemple : **`a { display: inline-block; }`** pour donner une largeur et une hauteur à un lien.

Le modèle de boîte : contenu, padding, border, margin




```css
.carte {
  width: 300px;
  padding: 20px;            /* intérieur */
  border: 2px solid #333;
  margin: 30px;             /* extérieur */
}
```

Les raccourcis de margin et padding

`10px` les 4 côtés

`10px 20px` haut et bas · gauche et droite

`10px 20px 30px 40px` haut, droite, bas, gauche (sens des aiguilles d'une montre)

`0 auto` centre un bloc qui a une largeur

```css
* { box-sizing: border-box; }
```

width inclut alors padding et border : la carte fait bien 300px, et non 300 + 40 + 4 = 344px.

Tailles, unités et débordement

Les unités

px absolue : ne change jamais

% par rapport au parent

em par rapport à la police du parent

rem par rapport à la police de la page (html)

vw / vh % de la largeur / hauteur de la fenêtre

```
div {
  width: 50%;
  height: 100px;
}
```

Limiter une taille

```
.container {
  width: 90%;
  min-width: 320px;
  max-width: 1100px;
  margin: 0 auto;
}
```

Le bloc suit la fenêtre (90 %) sans jamais descendre sous 320px ni dépasser 1100px, et reste centré.

C'est le conteneur classique d'une page.

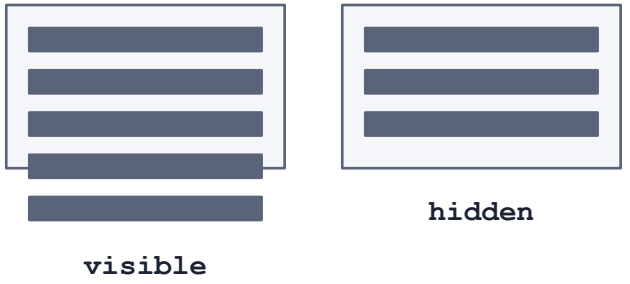
Quand ça déborde : overflow

visible par défaut : le contenu sort du bloc

hidden ce qui dépasse est coupé

scroll barres de défilement, toujours

auto barres seulement si nécessaire



Mettre en forme le texte

Propriété	Exemple	Effet
`font-size`	`16px` · `1.5em`	taille du texte
`font-family`	`"Open Sans", sans-serif`	police + police de secours
`font-weight`	`bold` · `700`	gras
`font-style`	`italic`	italique
`text-decoration`	`underline` · `none`	souligné / sans soulignement
`text-align`	`left` · `center` · `justify`	alignement
`line-height`	`1.6`	interligne
`color`	`#333333`	couleur du texte

Une police personnalisée

Avec Google Fonts, dans le <head> :

```
<link rel="stylesheet"
href="https://fonts.googleapis.com/c
ss2?family=Open+Sans">
```

Ou avec un fichier de police :

```
@font-face {
  font-family: "MaPolice";
  src: url("../fonts/
    mapolice.woff2");
}
p { font-family: "MaPolice"; }
```

font-family : guillemets si le nom contient des espaces, et toujours finir par une famille générique (serif, sans-serif ou monospace) au cas où la police ne se charge pas.

Couleurs et fonds

Écrire une couleur



red
un des 140 noms de couleur



#2a9c9f
hexadécimal : rouge, vert, bleu



rgb(42, 156, 159)
rouge, vert, bleu de 0 à 255



rgba(0, 0, 0, 0.5)
pareil + transparence de 0 à 1

Les fonds

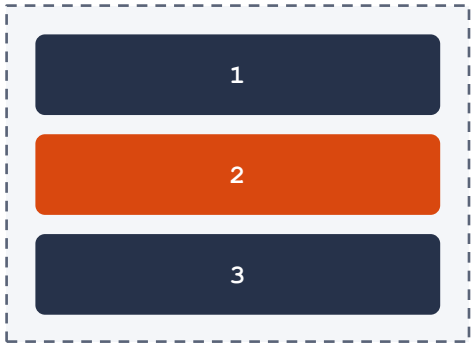
```css
.hero {
  color: white;                          /* texte */
  background-color: #34b1b3;             /* fond */
  background-image: url("../img/fond.jpg");
  background-repeat: no-repeat;
  background-size: cover;
  background-position: center;
}
```

no-repeat l'image ne se répète pas (repeat par défaut)

cover l'image couvre tout le bloc sans être déformée

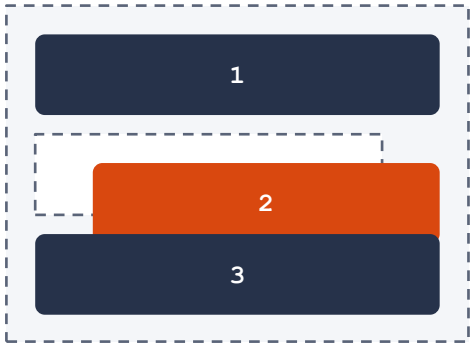
Piège : le chemin de url() part du fichier CSS, pas du HTML. Depuis css/style.css, on écrit **`url("../img/fond.jpg")`**.

La propriété position



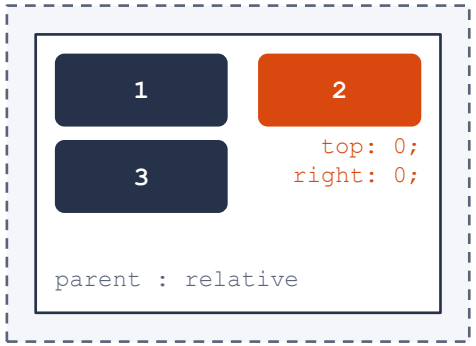
`position: static`

Par défaut. L'élément suit le flux normal de la page.



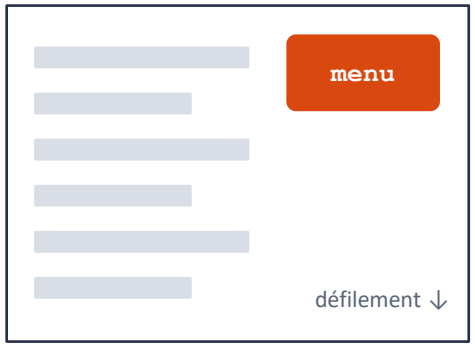
`position: relative`

Décalé par rapport à sa place d'origine (ici top: 15px; left: 30px). Sa place reste réservée.



`position: absolute`

Sort du flux. Placé par rapport au premier parent positionné (relative).



`position: fixed`

Placé par rapport à la fenêtre. Reste visible quand on fait défiler.

`top, right, bottom, left` déplacent l'élément (sauf en static). **`z-index`** : la plus grande valeur passe devant. **`float`** : sert seulement à faire couler du texte autour d'une image ; pour la mise en page, on utilise Flexbox ou Grid.

Flexbox et Grid en bref

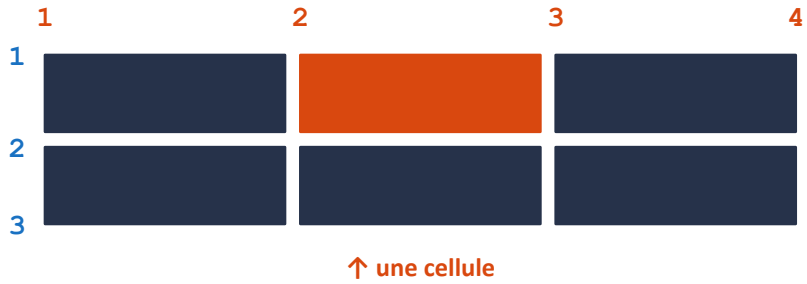
Flexbox : sur une ligne ou une colonne




```
#conteneur {
  display: flex;
  flex-direction: row;        /* ou column */
  flex-wrap: wrap;            /* à la ligne */
  justify-content: center;    /* axe principal */
  align-items: center;        /* axe secondaire */
}
```

Le conteneur est en display: flex ; ses enfants directs se rangent. Idéal pour une barre de menu ou une rangée de cartes.

Grid : en lignes et en colonnes



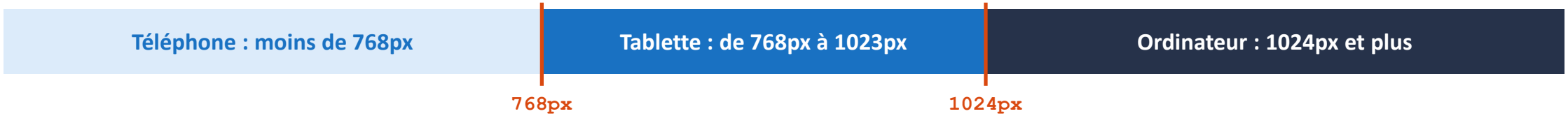

```
.grille {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 16px;
}
```

ligne les traits numérotés · piste : l'espace entre deux lignes

cellule une case · zone : plusieurs cases

Les media queries

Une media query applique des règles CSS **seulement si** une condition sur l'écran est vraie. Les points de rupture (breakpoints) sont les largeurs où l'affichage change.



Exemple : mobile d'abord

```css
/* 1. Styles du mobile, par défaut */
.colonnes { display: block; }

/* 2. À partir de 768px de large */
@media screen and (min-width: 768px) {
  .colonnes { display: flex; }
}
```

La syntaxe

type all, screen (écrans), print (impression)

condition min-width, max-width, orientation…

opérateurs and (et), not (non), virgule (ou)

Indispensable dans le <head> :

```
<meta name="viewport" content="width=device-width,
initial-scale=1.0">
```

Les étapes pour styliser une page

1 Créer et relier

Créer css/style.css et l'ajouter dans le <head> avec <link>.

2 Remettre à zéro

```
* { margin: 0; padding: 0; box-sizing: border-box; }
```

3 Styles généraux

Sur body : police, taille, couleur du texte, couleur de fond.

4 Le texte

Titres, paragraphes, liens (et leur état :hover).

5 Les boîtes

Largeurs, marges, bordures et fonds des blocs.

6 La mise en page

Conteneur centré, puis Flexbox, Grid ou position.

7 Le responsive

Media queries pour tablette et téléphone.

8 Vérifier

Inspecteur du navigateur (F12), validateur CSS du W3C, test en réduisant la fenêtre.

Avancer étape par étape et rafraîchir le navigateur après chaque modification : une erreur se trouve tout de suite.

10 · S'ENTRAÎNER

À vous de jouer

1

CSS Diner

Un jeu pour maîtriser les sélecteurs : balises, classes, combinateurs, pseudo-classes.

flukeout.github.io

2

Flexbox Froggy

Placer des grenouilles avec justify-content, align-items et flex-direction.

flexboxfroggy.com

3

Grid Garden

Arroser un potager avec grid-column, grid-row et grid-template-columns.

cssgridgarden.com

Documentation : developer.mozilla.org/fr **Compatibilité :** caniuse.com

Support réalisé par Harouna Kane